**CSED211 컴퓨터 시스템** 2026 가을 · 중간고사 10/29 (목) ·

# Lecture 2 · Bits, Bytes, Integers(비트, 바이트, 정수)

9/10 (목) · 슬라이드 "Lecture 2 Bits, Bytes, Integers" + 수업 녹취 정리 · **Data Lab 1의 직접 기반**

## 1 Bit와 Byte

### Everything is Bits

> **bit** = 0 또는 1, 저장 가능한 가장 작은 정보 단위 (논리적으로 0 = False, 1 = True). 비트 묶음을 어떻게 **encoding · 해석**하느냐에 따라 숫자, 집합, 문자열, 명령어까지 표현한다.

- 왜 2진수인가 → **전기적 구현이 쉽기 때문**
  - 아날로그 전압을 low 구간 / high 구간으로 나눠 0 · 1로 해석
  - range로 판정하므로 노이즈 · 전압 흔들림이 있어도 안정적으로 decode 가능
  - 이후로는 아날로그 신호는 잊고 디지털 0/1만 다룸

---

### Binary와 Powers of Two

- 10진수: 자리마다 10가지, 9 + 1 → 0 쓰고 carry 1 / 2진수: 자리마다 2가지, 01 + 1 → 10 (carry)
- 자리 $i$의 weight는 $2^i$. $1011_2 = 8 + 2 + 1 = 11$, $2^{10} = 1024 \approx 10^3$
- **2의 거듭제곱은 구구단처럼 외워둘 것 (교수 강조)**

---

$2^0 = 1,\ 2^1 = 2,\ 2^2 = 4,\ 2^3 = 8,\ 2^4 = 16,\ 2^5 = 32,\ 2^6 = 64,\ 2^7 = 128,\ 2^8 = 256,\ 2^9 = 512,\ 2^{10} = 1024$

$2^{10} \to \mathrm{K},\ 2^{20} \to \mathrm{M},\ 2^{30} \to \mathrm{G},\ 2^{40} \to \mathrm{T},\ 2^{50} \to \mathrm{P},\ 2^{60} \to \mathrm{E}$ · 큰 지수는 10단위로 쪼개서 계산: $2^{14} = 2^4 \cdot 2^{10} = 16\mathrm{K}$, $2^{32} = 2^2 \cdot 2^{30} = 4\mathrm{G}$

---

### Byte와 Hexadecimal

- **byte** = 8 bits, **프로그램이 다루는 최소 단위** (비트 하나는 그 자체로 쓸 일이 거의 없음)
  - binary 00000000 ~ 11111111 = decimal 0 ~ 255 = hex 0x00 ~ 0xFF
  - $2^8 = 256$가지, 0부터 세므로 최댓값은 $2^8 - 1 = 255$

- **hexadecimal**: base-16, 기호 0~9, A~F (10 = A, ⋯, 15 = F)
  - 2진수는 길고 읽기 어려움 → 4비트 = 16진수 한 자리로 바로 변환되는 절충안
  - C에서는 `0x` 접두사, 대소문자 구분 없음: `0xFA1D37B` = `0xfa1d37b`

| Hex | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Binary | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

### C 데이터 타입 크기 (bytes)

| C 타입 | Typical 32-bit | Typical 64-bit | x86-64 |
|---|---|---|---|
| char | 1 | 1 | 1 |
| short | 2 | 2 | 2 |
| int | 4 | 4 | 4 |
| long | 4 | 8 | 8 |
| float | 4 | 4 | 4 |
| double | 8 | 8 | 8 |
| long double | – | – | 10/16 |
| pointer (word) | 4 | 8 | 8 |

- 타입 크기가 저장 가능한 값의 범위를 결정 → 플랫폼이 몇 바이트를 주는지 알아둘 것
- int는 보통 4바이트, 양수 · 음수 모두 표현. 음수가 필요 없으면 unsigned int → 같은 4바이트로 더 큰 양수 범위
- **int ↔ unsigned 사이를 오가며 생기는 미묘한 버그가 이번 강의의 핵심 주의점**

## 2 Bitwise Operation

### Boolean Algebra

- 논리의 대수적 표현, True = 1, False = 0

| A | B | A & B (AND) | A \| B (OR) | A ^ B (XOR) | ~A (NOT) |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 |

- AND: 둘 다 1일 때만 1 · OR: 하나라도 1이면 1 · XOR: 정확히 하나만 1일 때 1 · NOT: 단항(unary) 연산, 반전

---

### Bit Vector 연산

- bitwise: **같은 위치의 비트끼리** 대응 연산 적용 (location-wise)

```
  01101001      01101001      01101001      ~ 01010101
& 01010101    | 01010101    ^ 01010101    ──────────
──────────    ──────────    ──────────      10101010
  01000001      01111101      00111100         = 0xAA
= 0x69&0x55   = 0x69|0x55   = 0x69^0x55
= 0x41        = 0x7D        = 0x3C
```

> **De Morgan 법칙**
> $\sim(a \,\&\, b) = \sim a \mid \sim b \qquad \sim(a \mid b) = \sim a \,\&\, \sim b$ 허용 연산이 제한된 Data Lab에서 연산을 서로 바꿔 쓰는 핵심 도구.

---

### 비트 벡터로 집합 표현 (bit flagging)

- $w$-bit 벡터 = $\{0, 1, \ldots, w-1\}$의 부분집합, $a_j = 1 \Leftrightarrow j \in A$
  - `01101001` → $\{0, 3, 5, 6\}$ (오른쪽 끝이 위치 0) · `01010101` → $\{0, 2, 4, 6\}$
- `a & b` = 교집합 $\{0, 6\}$ · `a | b` = 합집합 $\{0, 2, 3, 4, 5, 6\}$ · `a ^ b` = 대칭차 $\{2, 3, 4, 5\}$ · `~a` = 여집합 $\{1, 2, 4, 7\}$
- **bit flagging**: 각 비트 위치에 하나의 속성을 배정하고 켜짐/꺼짐으로 기록
  - 고수준 예: 걸을 수 있음 / 수영 가능 / 날 수 있음 … 을 비트별로 저장

  - 저수준 예: overflow 발생 여부, 값이 0인지, 값이 write됐는지 등 하드웨어 상태 플래그
  - 여러 대상의 벡터에 AND를 걸면 "모두 만족하는 속성"을 한 번에 구함

---

### C의 Bitwise Operator vs Logical Operator

- `& | ~ ^` (bitwise): 모든 integral 타입(long, int, short, char, unsigned …)에 적용, 인자를 비트 벡터로 보고 bitwise 적용
  - `~0x41` = ~01000001 → 10111110 = `0xBE` · `~0x00` → `0xFF`
  - 여기서는 CS:APP식 8비트 벡터 예시 — C의 int 식에서는 int 전체 비트가 반전됨 (하위 8비트만 보려면 `& 0xFF`로 마스킹)
  - hex ↔ binary 변환 요령: 4비트씩 잘라 각각 변환 (0x41 → 0100 | 0001)
- `&& || !` (logical): 값 전체를 True/False 하나로 축약: **0만 False, 0이 아니면(양수든 음수든) True**, 결과는 항상 0 또는 1
  - `!0x41` → 0x00 · `!0x00` → 0x01 · `!!0x41` → 0x01 (원래 값이 아니라 1!)
  - `0x69 && 0x55` → 0x01 · `0x69 || 0x55` → 0x01

---

`&&` vs `&`, `||` vs `|` 혼동: `0x69 & 0x55 = 0x41` 이지만 `0x69 && 0x55 = 0x01`. `!!x`는 원래 값이 아니라 0 또는 1이다.

---

### Shift 연산

| | |
|---|---|
| **left shift** `x << y` | 왼쪽으로 y칸 이동, 넘친 비트는 버림, 오른쪽은 0으로 채움 |
| **right shift** `x >> y` | 오른쪽으로 y칸 이동, 넘친 비트는 버림 |
| **logical shift** | 왼쪽을 0으로 채움 (값의 의미는 고려 안 함) |
| **arithmetic shift** | 왼쪽을 MSB 복제로 채움 |

- 수치적 의미: 왼쪽 1칸 = ×2, 왼쪽 k칸 = $\times 2^k$ / 오른쪽 1칸 = ÷2
  - signed 정수에서 MSB는 음수 weight → 음수를 2로 나눠도 음수여야 하므로 MSB가 1이면 1로 채워야 함 (arithmetic shift의 이유)

- C 표준 기준: unsigned right shift는 logical(0 채움) · 음수의 signed right shift는 구현 정의(일반적인 x86은 arithmetic) · signed left shift는 피연산자가 음수이거나 결과를 해당 signed 타입으로 표현할 수 없으면 UB

**비트 연산 — Bitwise / Logical Operations**

가상 비트 폭(w) 8비트

이동 거리(k) 1

~A | !A | A & B | A | B | A ^ B | A << k | A >> k (논리) | A >> k (산술)

## 입력 비트 — MSB → LSB

A · 비트를 클릭하여 변경

| 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|---|---|

**0xA5 · unsigned 165 · signed -91**

B · 비트를 클릭하여 변경

| 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|

**0x3C · unsigned 60 · signed 60**

## 결과 비트 — Result

| 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|

**0x24 · unsigned 36 · signed 36**

## 계산식 — 가상 w비트 모델

A & B = 0x24 · unsigned 36 · signed 36

① 가상 비트 모델 — Virtual w-bit Model
AND: 두 입력 비트가 모두 1일 때 1입니다.
폭 8비트의 패턴에 위젯 자체 규칙을 적용합니다.

② C 언어 표준 — Integer Promotion / Undefined Behavior
가상 w=4/8은 실제 C의 int 비트 폭이 아닙니다.
일반적인 32비트 int 환경에서 char/int8_t는 시프트 전에 int로 승격됩니다.
예: uint8_t a=0x80; a << 1의 계산 결과는 int 256입니다. 가상 8비트 왼쪽 시프트 결과 0x00과 다릅니다.
단, 결과를 다시 uint8_t에 저장하면 0x00으로 변환됩니다.
음수 시프트 횟수는 undefined behavior(UB)입니다.
32비트 int의 경우 k≥32도 UB입니다. char/int8_t 식이라고 해서 k≥8부터 UB가 되는 것은 아닙니다.
음수 signed 값을 왼쪽 시프트하는 경우에도 언어 버전과 값의 표현 가능성에 주의해야 합니다.

```
③ 일반적 x86 하드웨어 — Shift Instructions
x86의 32비트 시프트 명령은 횟수의 하위 5비트를 사용합니다(k & 31).
예: 기계어에서 32비트 값을 32번 시프트하면 실효 횟수는 0입니다.
그러나 C 식에서 k=32인 시프트는 UB이므로, 하드웨어의 마스킹 규칙을 C 결과로 가정하면 안 됩니다.
SAR는 부호 비트를 복제해 음의 무한대 방향으로 내림합니다. 이는 C 정수 나눗셈의 0 방향 절삭과 구별됩니다.
```

### Undefined behavior

shift 양이 **0보다 작거나 word size 이상**이면 undefined → 기계마다 다름 (exception을 내기도, 음수를 아주 큰 양수로 해석하기도). shift는 보수적으로 사용.

---

### 손계산 1 · Shift 결과

| 연산 | a = 01100010 | b = 10100010 |
|---|---|---|
| `x << 3` | 00010000 | 00010000 |
| `x >> 2` (logical) | 00011000 | 00101000 |
| `x >> 2` (arithmetic) | 00011000 | **11101000** |

a는 MSB가 0 → logical과 arithmetic 결과 동일. b는 MSB가 1 → arithmetic은 왼쪽 2칸을 1로 채움. `<< 3`은 왼쪽 3비트(011 / 101)가 범위를 벗어나 사라지고 오른쪽에 000이 들어옴.

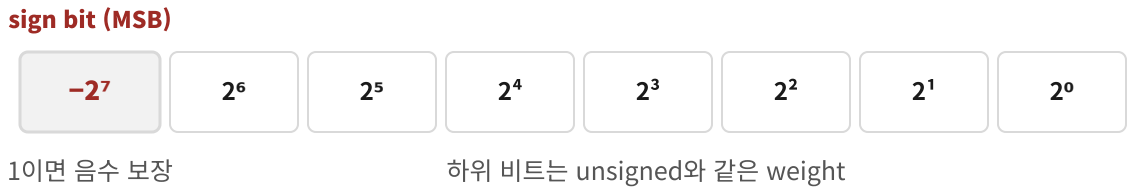
## 3 Integer Representation: Unsigned와 Two's Complement

### Encoding 공식

**unsigned: 모든 위치가 양의 weight, 켜진 비트의 weight를 합산** $B2U_w(\vec{x}) \;=\; \sum_{i=0}^{w-1} x_i\, 2^i$

**two's complement: 하위 비트 weight는 unsigned와 동일, MSB만 음의 weight**

$$B2T_w(\vec{x}) \;=\; -x_{w-1}\, 2^{w-1} \;+\; \sum_{i=0}^{w-2} x_i\, 2^i$$




w = 8 예시: 각 자리의 weight — MSB만 음수(−128), 나머지는 양수

Two's complement의 weight 배치 (슬라이드 그림 재구성)

- **sign bit** = MSB: 0이면 nonnegative, 1이면 negative
  - MSB 하나만으로 음수가 보장되는 이유: $\sum_{i=0}^{w-2} 2^i = 2^{w-1} - 1$
  - 하위 비트를 전부 켜도($2^{w-1} - 1$) MSB의 절댓값보다 1 작음 → MSB가 켜지면 무조건 음수

---

### 왜 Sign-Magnitude가 아니라 Two's Complement인가

- 직관적 대안: MSB = 부호, 나머지 = 크기 (예 4비트: 0011 = +3, 1011 = −3)
- **문제 ①** 0000 = +0, 1000 = −0 → **0이 두 개**, 패턴 낭비
- **문제 ②** 덧셈 하드웨어가 복잡: 부호가 같으면 더하고, 다르면 크기 비교 → 큰 쪽에서 작은 쪽을 빼고 큰 쪽 부호 → adder와 별도 subtractor 필요
- two's complement: 양수 · 음수 조합 상관없이 **같은 adder로 그냥 더하면 정답** → 하드웨어 면적 · 에너지 절약
  - 뺄셈도 $x - y = x + (-y)$, $-y = \sim y + 1$ → subtractor 불필요

---

### 손계산 2 · 15213과 −15213 (w = 16, short)

- x = 15213: hex `3B 6D` , binary `00111011 01101101`
- y = −15213: hex `C4 93` , binary `11000100 10010011`
- weight별 합산(켜진 비트만): y는 $1 + 2 + 16 + 128 + 1024 + 16384 - 32768 = -15213$
- 검산: $15213 + (-15213) = 0$. 두 패턴을 더하면 carry가 끝까지 밀려 나가며 0000 0000

### Range

**네 기준 값: 반드시 암기**

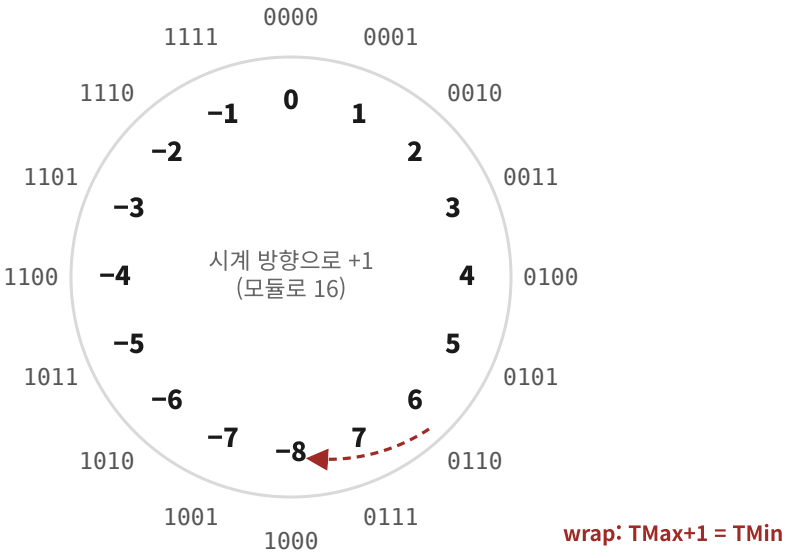
$\mathrm{UMin} = 0\ (00\ldots00)$ $\mathrm{UMax} = 2^w - 1\ (11\ldots11)$

$\mathrm{TMin} = -2^{w-1}\ (10\ldots00)$ $\mathrm{TMax} = 2^{w-1} - 1\ (01\ldots11)$ $-1 = 11\ldots11$

| w | 8 | 16 | 32 | 64 |
|---|---|---|---|---|
| **UMax** | 255 | 65,535 | 4,294,967,295 | 18,446,744,073,709,551,615 |
| **TMax** | 127 | 32,767 | 2,147,483,647 | 9,223,372,036,854,775,807 |
| **TMin** | −128 | −32,768 | −2,147,483,648 | −9,223,372,036,854,775,808 |

- w = 16: UMax = 65535 (FF FF), TMax = 32767 (7F FF), TMin = −32768 (80 00), −1 = FF FF, 0 = 00 00
  - 슬라이드 16의 TMin "−32767"은 오타, 정답은 −32768
- 관찰
  - 비대칭: $|\mathrm{TMin}| = \mathrm{TMax} + 1$ → 0이 양수 쪽 패턴(00…0)을 쓰므로 음수가 하나 더 많음
  - $\mathrm{TMax} + 1 = \mathrm{TMin}$ (4비트: 0111을 왼쪽 shift → 1110, +1 → 1111 … 0111 + 1 = 1000)
  - −1은 항상 all ones
- C: `#include <limits.h>` → ULONG_MAX, LONG_MAX, LONG_MIN 등, 값은 플랫폼마다 다름

### 손계산 3 · 4비트 전체 매핑




4비트 two's complement 수 원: 0111(TMax=7) 다음에 1000(TMin=−8)으로 wrap (슬라이드 그림 재구성)

| X | B2U(X) | B2T(X) | X | B2U(X) | B2T(X) |
|---|---|---|---|---|---|
| 0000 | 0 | 0 | 1000 | 8 | −8 |
| 0001 | 1 | 1 | 1001 | 9 | −7 |
| 0010 | 2 | 2 | 1010 | 10 | −6 |
| 0011 | 3 | 3 | 1011 | 11 | −5 |
| 0100 | 4 | 4 | 1100 | 12 | −4 |
| 0101 | 5 | 5 | 1101 | 13 | −3 |
| 0110 | 6 | 6 | 1110 | 14 | −2 |
| 0111 | 7 | 7 | 1111 | 15 | −1 |

- 0000~0111: unsigned와 signed가 같은 값 (**equivalence**: 양수 encoding 동일)
- 1000에서 signed는 −8로 큰 점프, 이후 1씩 증가해 1111 = −1
- **uniqueness**: 모든 비트 패턴은 정확히 하나의 정수, 표현 가능한 정수는 정확히 하나의 패턴 (0도 하나뿐)

- 역함수 존재: $U2B_w = B2U_w^{-1}$, $T2B_w = B2T_w^{-1}$ → 패턴 ↔ 값 결정적으로 왕복 가능

## 4 Conversion과 Casting

### Signed ↔ Unsigned Mapping

**비트 패턴은 그대로, 해석만 바꾼다.** 하드웨어는 signed/unsigned를 신경 쓰지 않고 비트에 같은 연산만 적용. MSB의 weight가 $-2^{w-1} \to +2^{w-1}$로 바뀜 → 차이 $2^w$.

$$T2U_w(x) = \begin{cases} x & x \geq 0 \\ x + 2^w & x < 0 \end{cases} \qquad U2T_w(u) = \begin{cases} u & u \leq \text{TMax} \\ u - 2^w & u > \text{TMax} \end{cases}$$

- "Large negative weight becomes large positive weight" → 음수가 갑자기 큰 양수가 되는 **의미 붕괴**
- 4비트에서는 1000~1111 구간이 ±16 차이 (−8 ↔ 8, −7 ↔ 9, ⋯, −1 ↔ 15)

#### 손계산 4 · T2U / U2T

| w | 원래 값 | 비트 | 재해석 | 계산 |
|---|---|---|---|---|
| 4 | T: −5 | `1011` | U: 11 | −5 + 16 = 11 |
| 4 | U: 9 | `1001` | T: −7 | 9 − 16 = −7 |
| 8 | T: −42 | `11010110` | U: 214 | −42 + 256 = 214 |
| 16 | T: −15213 | `C4 93` | U: 50323 | −15213 + 65536 |
| 32 | T: −1 | `FF FF FF FF` | U: 4294967295 (UMax) | $-1 + 2^{32}$ |
| 32 | T: TMin | `80 00 00 00` | U: 2147483648 | $-2^{31} + 2^{32} = 2^{31}$ |

### C에서의 Signed / Unsigned

- 상수: 기본은 signed, 접미사 `U`가 붙으면 unsigned (`0U`, `4294967259U`)
- explicit casting: T2U / U2T와 동일 (비트 유지) · implicit casting: 대입, 함수 호출에서도 자동 발생

```
int tx, ty;
unsigned ux, uy;
tx = (int) ux;      // explicit
uy = (unsigned) ty; // explicit
tx = ux;            // implicit (대입)
uy = ty;            // implicit (대입)
```

- C는 이런 변환의 책임을 프로그래머에게 넘김

---

### Casting Surprise: 가장 많이 틀리는 규칙

한 식 안에 signed와 unsigned가 섞이면 **signed가 unsigned로 implicit cast**. 산술 연산뿐 아니라 **비교 연산** `< > == <= >=`에도 적용!

| 상수 1 | 관계 | 상수 2 | 결과 | 해석 · 이유 |
|---|---|---|---|---|
| 0 | == | 0U | 1 | unsigned: 둘 다 0 |
| −1 | < | 0 | 1 | signed: 평범한 비교 |
| −1 | > | 0U | **1** | unsigned: −1 → UMax |
| 2147483647 | > | −2147483648 | 1 | signed: TMax > TMin |
| 2147483647U | < | −2147483648 | **1** | unsigned: TMin → 2147483648, 2147483647 < 2147483648 |
| −1 | > | −2 | 1 | signed: 평범한 비교 |
| (unsigned) −1 | > | −2 | **1** | unsigned: UMax > UMax − 1 |
| 2147483647 | < | 2147483648U | 1 | unsigned: $2147483647 < 2^{31}$ |
| 2147483647 | > | (int) 2147483648U | 1 | signed: (int)$2^{31}$ = TMin |

- 슬라이드 표에는 "−1 vs (unsigned) −2"와 "(int) 2147383648U"로 적혀 있음 (후자는 2147483648U의 오타로 보임). 어느 쪽이든 관계는 동일: (unsigned)−2 = 4294967294 < 4294967295 = (unsigned)−1 이므로 −1 쪽이 큼.
- 교수 강조: −1 vs 0U → 직관상 −1이 작지만 C에서는 −1이 UMax가 되어 더 큼

## 5 Extension과 Truncation

### Sign Extension

- 목표: $w$ 비트 signed $\vec{x}$를 **같은 값**을 갖는 $w+k$ 비트 $\vec{x}'$로
- 규칙: 부호 비트 $x_{w-1}$를 $k$개 복사해 앞에 붙임

$$\vec{x}' = (\underbrace{x_{w-1}, \ldots, x_{w-1}}_{k \text{ copies}}, x_{w-1}, x_{w-2}, \ldots, x_0)$$




sign extension: −128 + 64 + 32 + 16 + 8 + 2 + 1 = −5 (값 보존)

1011(−5) → 11111011(−5): 부호 비트 복사로 확장 (슬라이드 그림 재구성)

- zero extension: 앞을 0으로 채우면 값 유지 (전부 양의 weight)
- signed에서 음수를 0으로 채우면 → **양수로 바뀌어 의미 붕괴** → MSB 복사 필요
- C는 작은 타입 → 큰 타입 변환 시 자동으로 sign extension

### 손계산 5 · Sign Extension이 값을 보존하는 이유

- 4비트 1000(−8) → 5비트 11000: weight −16, 8, 4, 2, 1 → −16 + 8 = **−8**
- 기존 MSB는 −8에서 +8로 바뀌고, 새 MSB는 두 배 큰 −16 → 합 −8로 상쇄. 한 칸 늘릴 때마다 $-2^w + 2^{w-1} = -2^{w-1}$ → 귀납적으로 k칸도 성립
- 양수 0100(4) → 00100(4): 0을 복사하므로 당연히 유지
- 4비트 1011(−5) → 8비트 11111011: $-128 + 64 + 32 + 16 + 8 + 2 + 1 = -5$
- short → int 예 (슬라이드): x = 15213: `3B 6D` → `00 00 3B 6D` · y = −15213: `C4 93` → `FF FF C4 93`

```
short int x = 15213;
int ix = (int) x;   // 00 00 3B 6D
short int y = -15213;
int iy = (int) y;   // FF FF C4 93
```

---

### Truncation

- 넓은 타입 → 좁은 타입 (예: unsigned int → unsigned short): **상위 비트를 그냥 버림**, 남은 비트를 재해석
- 확장은 정보 손실 없음 / 절단은 **정보 손실 가능** → 정확성 유지는 프로그래머 책임 (하드웨어가 보장하지 않음)
- 의미: 버려지는 경계의 weight가 $2^k$ → **mod** $2^k$ 연산과 같은 효과
  - unsigned: $U2U_k(x) = x \bmod 2^k$
  - signed: mod와 비슷하지만 결과를 다시 two's complement로 해석 → 작은 수에서만 기대대로 동작

- **Expanding** (short → int): unsigned는 0 추가 (zero extension), signed는 sign extension → **항상 기대한 값**
- **Truncating** (int → short): unsigned는 상위 비트 버림 = mod, signed는 상위 비트 버림 후 재해석 (mod 유사) → **작은 수에서만** 기대한 값

**정수 표현 — Two's Complement**

비트 폭(w) 5

+1 (carry 관찰) | 부호 확장 w→w+4 | truncate w→4

### 비트 벡터 — MSB → LSB

| 0 | 0 | 1 | 1 | 1 |
|---|---|---|---|---|

Hex 0x07 · unsigned 7 · signed 7

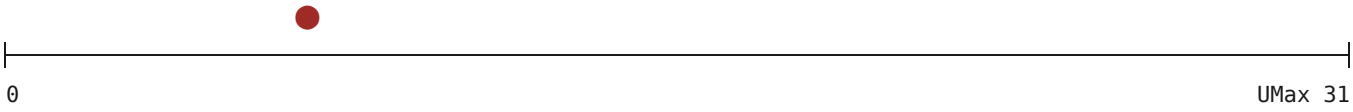
### 해석 공식 — B2U / B2T

**Unsigned (부호 없음):** $B2U = 0\times2^4 + 0\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0 = 7$

**Two's complement (2의 보수):** $B2T = -0\times2^4 + 0\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0 = 7$

### 숫자선 — Number Lines

**Unsigned · 현재 값 7**




**Two's complement · 현재 값 7**




TMin = -16 · TMax = 15 · UMax = 31
같은 비트 패턴도 unsigned / signed 해석에 따라 값이 다릅니다.
숫자선의 빨간 점은 현재 값이며, 경계 라벨은 트랙 안쪽에 표시됩니다.

## 손계산 6 · Truncation

- 8비트 → 4비트 (하위 4비트만 남김)
  - `11010110` (U: 214) → `0110` (U: 6): $214 \bmod 16 = 6$
  - `11010110` (T: −42) → `0110` (T: 6): **음수가 양수로 바뀜**
  - `11111011` (T: −5) → `1011` (T: −5): 작은 수 → 값 유지
- 32비트 → 16비트 (보충, CS:APP 예): int 53191 = `0x0000CFC7` → short `0xCFC7` = −12345
  - $53191 \bmod 65536 = 53191$, 이를 16비트 two's complement로 재해석 → $53191 - 65536 = -12345$

## 6 Integer Arithmetic

### Negation: ~x + 1

**Two's complement 부호 반전**

$-x \;=\; \sim x + 1$ 증명 스케치: $x + \sim x = -1$ (반전한 것과 더하면 모든 자리가 1, carry 없음)
→ 양변에서 $x$를 이항하면 $\sim x = -1 - x$.

- 교수 예 (4비트): $3 = 0011$, $\sim 3 = 1100$, $\sim 3 + 1 = 1101 = -3$
- x = 15213: `3B 6D` → ~x = −15214 ( `C4 92` ) → ~x + 1 = −15213 ( `C4 93` )
- x = 0: ~x = −1 ( `FF FF` ) → ~x + 1 = 0 (carry 버림)
  - 슬라이드 31의 x = 0 행 Hex "3B 6D"는 오타, 00 00이 맞음
- 의의: 뺄셈 $x - y = x + \sim y + 1$ → **별도 subtractor 없이 adder 하나로 처리**

---

$\sim \mathrm{TMin} + 1 = \mathrm{TMin}$ (4비트: 1000 → 0111 + 1 = 1000). TMin의 절댓값은 표현할 수 없다. 절댓값·부호 반전 문제에서 TMin은 항상 특수 케이스.

---

### Unsigned Addition

- $w$ 비트 두 수의 참 합은 최대 $w + 1$ 비트 → carry를 버림 → 결과 $w$ 비트

$$u +_w^u v \;=\; (u + v) \bmod 2^w$$

- MSB 밖으로 carry가 나가면 = **unsigned overflow** (결과가 참 합보다 $2^w$ 작음)

### Two's Complement Addition

- **TAdd와 UAdd는 비트 수준에서 동일** → 같은 adder 사용
- 적용 범위: TMax+1 = TMin 같은 wrap 규칙은 비트 벡터/기계 모델의 항등식 — C의 unsigned 연산은 모듈로로 보장되지만 signed overflow는 C 표준상 UB

```
int s, t, u, v;
s = (int) ((unsigned) u + (unsigned) v);
t = u + v;
// 항상 s == t
```

적용 범위: 동일한 폭의 int와 unsigned를 가정하고 `u + v`가 signed 범위 안에 있으면 두 결과는 일치 · signed overflow가 발생하면 C 표준상 UB라 `s == t`를 모든 구현에서 항상 참이라고 단정할 수는 없음

- 참 합 $w+1$ 비트 → MSB(carry) 버림 → 남은 $w$ 비트를 two's complement로 해석
- **overflow 가능성은 부호 조합으로 판단 (교수 강조)**
  - 양수 + 양수: 가능 (**PosOver**), 합 $\geq 2^{w-1}$ → MSB가 1이 되어 음수로 보임 (결과 $= u+v-2^w$)
  - 음수 + 음수: 가능 (**NegOver**), 합 < $-2^{w-1}$ → 양수로 보임 (결과 $= u+v+2^w$)
  - 양수 + 음수: **불가능**, 결과가 항상 두 피연산자 사이 → 범위 안

$$u +_w^t v \;=\; U2T_w\big(u +_w^u v\big) \;=\; U2T_w\big((u+v) \bmod 2^w\big)$$

- carry가 나갔다고 signed 결과가 틀린 것은 아님 (예: −1 + −1 = 1111 + 1111 = 1 1110 → 1110 = −2, 정상)

**손계산 7 · 4비트 덧셈과 Overflow**

| u | v | 비트 합 | 4비트 결과 | 해석 | 판정 |
|---|---|---|---|---|---|
| U: 13 | U: 5 | 1101 + 0101 = 10010 | 0010 | U: 2 | unsigned overflow, 18 − 16 = 2 |
| T: 4 | T: 4 | 0100 + 0100 = 1000 | 1000 | T: −8 | PosOver, 8 − 16 = −8 |
| T: −8 | T: −5 | 1000 + 1011 = 10011 | 0011 | T: 3 | NegOver, −13 + 16 = 3 |
| T: 3 | T: 4 | 0011 + 0100 = 0111 | 0111 | T: 7 | 정상 (양수+양수여도 overflow 아닐 수 있음) |
| T: 7 | T: −3 | 0111 + 1101 = 10100 | 0100 | T: 4 | 정상 (이부호, carry는 버림) |
| T: −1 | T: −1 | 1111 + 1111 = 11110 | 1110 | T: −2 | 정상 |

**판별법:** 같은 부호끼리 더했는데 결과의 MSB가 바뀌면 overflow.

### Multiplication

- $w$ 비트 × $w$ 비트의 정확한 결과는 $2w$ 비트를 넘을 수 있음
  - Unsigned 최대: $2w$ 비트, $(2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$
  - Two's comp. 최소: $2w - 1$ 비트, $-2^{w-1}(2^{w-1} - 1) = -2^{2w-2} + 2^{w-1}$
  - Two's comp. 최대: $2w$ 비트 ($\mathrm{TMin}^2$일 때만), $(-2^{w-1})^2 = 2^{2w-2}$
- 정확한 값을 유지하려면 곱할 때마다 word size를 계속 늘려야 함 (정확한 값이 필요하면 arbitrary-precision으로)
- C의 표준 곱셈: **상위 $w$ 비트를 버림** → 덧셈(1비트 버림)보다 버리는 양이 많아 overflow가 훨씬 흔함

$$u *_w v \;=\; (u \cdot v) \bmod 2^w$$

- signed 곱셈도 상위 $w$ 비트를 버림 → 상위 비트는 signed/unsigned가 다를 수 있지만 **하위 $w$ 비트는 동일**
- 일부 아키텍처는 $2w$ 비트 결과를 상위/하위로 나눠 다른 레지스터에 보관 → 활용은 프로그래머 책임
- 곱셈기 하드웨어는 구현이 복잡하고 느림 → **가능한 한 shift로 대체**

---

**손계산 8 · 4비트 곱셈, 하위 비트는 같다**

- u = 1011, v = 0011
  - Unsigned: u = 11, v = 3, 참 곱 33 = 0010 0001 → 하위 4비트 0001 → 결과 1 ($33 \bmod 16 = 1$)
  - Two's comp.: u = −5, v = 3, 참 곱 −15 = 1111 0001 → 하위 4비트 0001 → 결과 1
- 상위 4비트(0010 vs 1111)는 다르지만 버려지는 부분, 남는 하위 4비트는 동일
- signed 예: $3 \times 5 = 15$ = 1111 → 4비트 signed로 재해석하면 −1 (overflow)

---

**Shift로 곱하기 / 나누기**

$$u \ll k \;=\; u \cdot 2^k \bmod 2^w \qquad u \gg k \;=\; \lfloor u/2^k \rfloor$$

- `u << 3 == u * 8` · `(u << 5) - (u << 3) == u * 24` (24 = 32 − 8)
- 대부분의 기계에서 shift · add가 곱셈보다 빠름 → 컴파일러가 상수 곱셈을 자동으로 shift 조합으로 최적화. 한계: 단독 shift는 2의 거듭제곱 배만 가능
- unsigned 나누기: `u >> k`, logical shift 사용. 오른쪽으로 밀려난 비트 = 소수점 아래 → 버려짐 → **floor 효과**
  - 교수 예: 0110(6) >> 1 = 0011(3), 0011(3) >> 1 = 0001(1 = $\lfloor 3/2 \rfloor$)
  - x = 15213 ( `3B 6D` ): x >> 1 = 7606 ( `1D B6` ) · x >> 4 = 950 ( `03 B6` ) · x >> 8 = 59 ( `00 3B` )
- signed: arithmetic shift (MSB 복제) → 음수가 음수로 유지. sign extension과 같은 직관: weight가 반으로 줄면서 부호 정보가 사라지는 것을 두 배 큰 음의 weight로 보충

### 손계산 9 · 음수 Right Shift와 Bias

- x = −4 (1100): x >> 1 = 1110 = −2, C의 x / 2 = −2 → 일치
- x = −5 (1011): x >> 1 = 1101 = −3, C의 x / 2 = −2 → **불일치**
- $\lfloor -5/2 \rfloor = -3$: shift는 $-\infty$ 방향, 정수 나눗셈은 0 방향으로 버려 −2를 기대
- **bias**: 음수일 때 shift 전에 더해 0 방향 반올림으로 맞춤. k = 1이면 bias = 1: $(x + 1) \gg 1$.
  일반적으로 $(x + 2^k - 1) \gg k$ (x < 0일 때 사용)

---

## Unsigned를 언제 쓰나

---

### 쓰지 말 것: 함의를 모른 채 unsigned 사용

실수하기 쉽고 매우 미묘하다:

```
unsigned i;
for (i = cnt-2; i >= 0; i--)   // i >= 0 은 항상 참 → 무한 루프
    a[i] += a[i+1];
```

- unsigned i가 0에서 1 감소 → UMax로 wrap around, 절대 0보다 작아지지 않음

```
#define DELTA sizeof(int)
int i;
for (i = cnt; i - DELTA >= 0; i -= DELTA)   // 역시 항상 참
    ...
```

- i는 signed지만 `sizeof`가 unsigned(size_t)를 반환 → `i - DELTA`가 unsigned로 계산 → 0 미만 불가

- 쓸 것: modular arithmetic (multi-precision 연산), 집합 표현 (logical right shift, sign extension 없음)

### Unsigned로 거꾸로 세기 (Robert Seacord, "Secure Coding in C and C++")

C 표준은 unsigned 덧셈이 modular arithmetic임을 보장: $(x + y) \bmod 2^w$.

```
unsigned i;
for (i = cnt-2; i < cnt; i--)   // 0 다음 UMax가 되면 i < cnt 가 거짓 → 종료
    a[i] += a[i+1];
```

더 좋은 방법: **size_t** (word size 길이의 unsigned, 64-bit에서 8바이트). cnt가 4바이트 unsigned의 UMax여도 index가 더 넓어서 동작:

```
size_t i;
for (i = cnt-2; i < cnt; i--)
    a[i] += a[i+1];
```

생각해볼 것: cnt가 signed이고 < 0 이면?

## 7 Memory에서의 표현

### Byte-Oriented Memory와 Machine Word

- 프로그램은 **virtual address** 사용: 메모리를 아주 큰 byte 배열로 간주 (실제와 다르지만 그렇게 생각해도 됨)
  - address = 배열 index, pointer 변수 = address를 저장
  - 시스템은 프로세스마다 private address space 제공. process = 실행 중인 프로그램. 다른 프로세스 데이터는 접근 불가 (OS의 보호, virtual memory, 이후 강의)
- **word size**: 정수 데이터의 명목상 크기, 더 정확히는 **address 크기** (일반적으로 가상 주소·포인터 폭과 일치 — 엄밀히는 CPU가 자연스럽게 처리하는 데이터 폭)
  - CPU 레지스터 폭과 일치 (32-bit 기계 = 32비트 레지스터), program counter 같은 레지스터가 메모리 주소를 가리킴
- 32-bit (4 bytes): 주소 공간 $2^{32}$ bytes = **4 GiB** · 64-bit: 이론상 $2^{64}$ bytes = **16 EiB** $\approx 1.8 \times 10^{19}$ bytes
  - 슬라이드에는 "18 PB, 18.4 × $10^{15}$"로 적혀 있으나 $2^{64} \approx 1.8 \times 10^{19}$이므로 18 EB가 맞음 (교수도 녹취에서 exabyte로 설명)
  - 실제로는 64비트 전부를 주소로 쓰지 않음 → 실용 한계 약 256 TB ($2^{48}$ bytes에 해당)
- address는 byte 위치를 지정, **word의 address = word의 첫 바이트 address**. 연속된 word의 address 차이: 4 (32-bit), 8 (64-bit)

  - 4바이트 word 주소는 하위 2비트가 항상 00, 8바이트 word 주소는 하위 3비트가 항상 000. 이 하위 비트들은 word 안에서 몇 번째 byte인지 offset으로 사용

---

### Byte Ordering · Endianness

> **Big Endian**: most significant byte가 가장 낮은 address (Sun, PowerPC Mac, IBM, Internet)
> **Little Endian**: least significant byte가 가장 낮은 address (x86, Android · iOS · Windows를 돌리는 ARM)
> 암기법: **L**ittle endian = **L**east significant byte at **L**owest address

**0x01234567 at 0x100**

| 주소 | 0x100 | 0x101 | 0x102 | 0x103 | |
|---|---|---|---|---|---|
| **Big Endian** | **01** | **23** | **45** | **67** | 사람이 쓰는 순서 그대로 |
| **Little Endian** | **67** | **45** | **23** | **01** | least significant byte(0x67)가 맨 앞 |

x86 = little · Internet = big → 송신 측에서 big으로 바꿔 보내고 수신 측에서 다시 little로 변환

같은 값 0x01234567의 메모리 배치: big endian vs little endian (슬라이드 그림 재구성)

- 정수의 실제 바이트 배치 (낮은 주소 → 높은 주소)
  - int a = 15213 ( `0x00003B6D` ): IA32/x86-64 (little) `6D 3B 00 00` · Sun (big) `00 00 3B 6D`
  - int b = −15213 ( `0xFFFFC493` ): IA32/x86-64 `93 C4 FF FF` · Sun `FF FF C4 93`
  - long c = 15213: IA32 `6D 3B 00 00` (4B) · x86-64 `6D 3B 00 00 00 00 00 00` (8B) · Sun `00 00 3B 6D`

---

### Byte-Reversed Listing 읽기

- disassembly: 기계어를 텍스트로 보여주는 것 (기계어를 읽는 프로그램이 생성)

```
8048365:  5b                      pop    %ebx
8048366:  81 c3 ab 12 00 00       add    $0x12ab,%ebx
804836c:  83 bb 28 00 00 00 00    cmpl   $0x0,0x28(%ebx)
```

- `0x12ab` 해독: 32비트로 padding 0x000012ab → byte 분리 00 00 12 ab → 역순 `ab 12 00 00` (little endian 기계어에 나타나는 형태)

---

### show_bytes와 String 표현

```
typedef unsigned char *pointer;
void show_bytes(pointer start, int len) {
    int i;
    for (i = 0; i < len; i++)
        printf("%p\t0x%.2x\n", start + i, start[i]);  // %p: pointer, %x: hex
    printf("\n");
}
int a = 15213;
show_bytes((pointer) &a, sizeof(int));
```

- pointer를 `unsigned char *`로 cast → byte 배열처럼 하나씩 접근. Linux 결과: 0x6d, 0x3b, 0x00, 0x00 → little endian 확인
- pointer 값 자체는 컴파일러 · 기계마다 다르고 실행 환경(ASLR, 메모리 배치)에 따라 달라질 수 있음. virtual → physical 매핑은 이후 강의
- C 문자열 = char 배열, 각 문자는 ASCII (7-bit 표준 encoding). 문자 '0'은 0x30, 숫자 i는 0x30 + i. **null-terminated**: 마지막 문자는 0x00
- **문자열에는 byte ordering 문제 없음**: 단위가 1바이트라서 little / big 상관없이 같은 위치 (endianness는 2·4·8바이트 단위일 때만 의미)
  - `char a[6] = "18243"` → Linux / Sun 모두: `31 38 32 34 33 00`

## 8 시험 포인트

- B2U, B2T 공식과 MSB의 음의 weight $-2^{w-1}$
- UMax $= 2^w - 1$, TMax $= 2^{w-1} - 1$, TMin $= -2^{w-1}$, −1 = all ones, $|\mathrm{TMin}| = \mathrm{TMax} + 1$, $\mathrm{TMax} + 1 = \mathrm{TMin}$
- w = 8 범위 값: 255, 127, −128
- w = 16 범위 값: 65535, 32767, −32768
- w = 32 범위 값: 4294967295, 2147483647, −2147483648
- 4비트 B2U / B2T 표를 즉시 쓸 수 있어야 함
- sign-magnitude의 두 문제(±0, adder 복잡) vs two's complement의 장점(adder 하나로 덧셈 · 뺄셈)
- T2U: 비트 유지, 음수면 $+2^w$
- signed · unsigned가 섞이면 signed → unsigned, 비교 연산 포함
- Casting surprises: −1 > 0U, 2147483647U < −2147483648, sizeof가 unsigned라는 점
- &, |, ^, ~ (bitwise) vs &&, ||, ! (logical, 결과 0/1)
- logical vs arithmetic right shift
- shift 양이 음수 또는 ≥ w면 undefined
- sign extension 규칙과 값이 보존되는 이유
- truncation = mod $2^k$ (signed는 재해석)
- $-x = \sim x + 1$, $\sim\mathrm{TMin} + 1 = \mathrm{TMin}$
- UAdd = mod $2^w$, TAdd는 UAdd와 비트 동일
- overflow는 같은 부호끼리 더할 때만 (PosOver → 음수, NegOver → 양수)
- 곱셈은 상위 w비트 버림, signed · unsigned의 하위 w비트 동일
- $u \ll k = u \cdot 2^k \bmod 2^w$, $u \gg k = \lfloor u/2^k \rfloor$, 음수 arithmetic shift는 −∞ 방향 → bias $2^k - 1$
- unsigned loop index 버그 (i >= 0 무한 루프), size_t 사용
- word address = 첫 byte 주소, 32-bit = 4 GiB

- big vs little endian 정의, 0x01234567 배치, x86 = little, Internet = big, 문자열은 endian 무관

## 9 연습문제

---

2-1–100의 8비트 표현★

8비트에서 −100의 비트 패턴, hex, 그리고 이를 unsigned로 해석한 값을 구하라.

**힌트 보기**

$-x = \sim x + 1$로 패턴을 만들고, unsigned 해석은 T2U ($+2^w$).

**정답 보기**

개념 정리는 힌트에 있음.

① 100을 2진수로: `0110 0100` (64 + 32 + 4).

> 먼저 절댓값의 패턴 확보

② 비트 반전 후 1을 더한다: ~100 = `1001 1011`, +1 → `1001 1100` = 0x9C.

> ~x + 1

③ 검산: $100 + 156 = 256 \equiv 0 \pmod{256}$.

④ unsigned 해석: T2U로 $-100 + 256 = 156$ (= 0x9C).

최종 답 **`1001 1100` = 0x9C, unsigned로 156**

2-24비트 덧셈과 overflow 판정★★

4비트 two's complement에서 다음을 계산하고 overflow 여부를 판정하라. ① 5 + 6 ② −7 + −2 ③ −6 + 5

**힌트 보기**

overflow는 같은 부호끼리 더할 때만 가능. 판정: 결과의 MSB가 피연산자와 달라졌는가.

**정답 보기**

① ① 0101 + 0110 = 1011 → −5. 참값 11 > TMax 7 → PosOver ($11 - 16 = -5$).

> 양수+양수인데 MSB=1

② ② 1001 + 1110 = 1 0111 → carry 버림 0111 = 7. 참값 −9 < TMin −8 → NegOver ($-9 + 16 = 7$).

> 음수+음수인데 MSB=0

③ ③ 1010 + 0101 = 1111 → −1. 이부호 덧셈이므로 overflow 불가.

최종 답 **① −5 (PosOver) ② 7 (NegOver) ③ −1 (정상)**

2-3signed/unsigned 비교★★

32비트 C에서 다음 비교의 결과(1 또는 0)와 그 이유를 쓰라. ① `-1 < 0U` ② `-1 > -2` ③ `(unsigned)-1 > -2`

**힌트 보기**

한쪽이라도 unsigned면 다른 쪽도 unsigned로 변환된다. −1의 unsigned 값은 UMax.

**정답 보기**

① ① 0U 때문에 −1이 unsigned로 변환 → 4294967295 < 0 은 거짓.

> 결과 0

② ② 둘 다 signed, 평범한 비교.

> 결과 1

③ ③ −2도 unsigned로 변환 → 4294967295 > 4294967294.

> 결과 1

**최종 답** **① 0 ② 1 ③ 1**

2-4확장과 절단★★

`short x = -3` 을 int로 변환했을 때의 hex 값과, `int y = 0x0001FFFD` 를 short로 변환했을 때의 값을 구하라.

**힌트 보기**

확장은 부호 비트 복사(값 보존), 절단은 상위 비트를 버리고 재해석(mod $2^k$ — 일반적인 2의 보수 환경 기준 (범위 밖 signed 변환은 C 표준상 구현 정의)).

**정답 보기**

① −3 (16비트) = 0xFFFD → sign extension → 0xFFFFFFFD = −3 (값 유지).

> 부호 비트 1을 복사

② 0x0001FFFD → 하위 16비트 0xFFFD만 남음 → short로 해석하면 −3.

> 상위 비트 버림

③ 확인: y의 원래 값은 131069, $131069 \bmod 65536 = 65533$, 이를 signed로 재해석하면 $65533 - 65536 = -3$ → truncation으로 값이 바뀜.

**최종 답** **0xFFFFFFFD (−3 유지), short로 −3 (값 바뀜)**

2-5음수의 시프트와 bias★★

4비트 signed x = 1001 (−7)에 대해 `x >> 2` (arithmetic)의 결과와, C의 `x / 4`와 같게 만드는 방법을 쓰라.

힌트 보기

arithmetic shift는 $-\infty$ 방향으로 버리고 C 나눗셈은 0 방향. 차이를 메우는 것이 bias $2^k - 1$.

정답 보기

① 1001 >> 2 = 1110 = −2 $= \lfloor -7/4 \rfloor$.

> MSB 1 복제로 채움

② C의 x / 4는 0 방향으로 버려 −1 기대 → 불일치 확인.

③ bias $2^k - 1 = 3$: $(-7 + 3) \gg 2 = -4 \gg 2 = -1$.

> 1100 >> 2 = 1111

최종 답 **`x >> 2` = 1110 = −2 · C와 맞추려면 `(x + 3) >> 2`**

2-6항상 참인가? (종합)★★★

32비트, two's complement, `int x, y; unsigned ux = x;` 일 때 각 명제가 항상 참인지 판정하라. 거짓이면 반례를 들라.

판정 기준: 32비트 two's complement 비트 모델(기계 동작) — C 표준상 signed overflow는 UB라, 실제 C 코드에서는 표준이 보장하지 않는 결과가 포함될 수 있음에 유의.

- ① x < 0 ⇒ x*2 < 0
- ② ux >= 0
- ③ x > 0 && y > 0 ⇒ x + y > 0
- ④ x >= 0 ⇒ -x <= 0
- ⑤ x <= 0 ⇒ -x >= 0
- ⑥ ux >> 3 == ux/8
- ⑦ x >> 3 == x/8

**힌트 보기**

반례 후보는 항상 극값에서: TMin, TMax, −1. shift 문제는 음수에서 floor vs 0 방향 차이를 본다.

**정답 보기**

① ① 거짓: x = TMin → x*2 = 0 (overflow).

> TMin×2는 mod $2^{32}$로 0

② ② 참: unsigned는 항상 0 이상.

③ ③ 거짓: x = TMax, y = 1 → TMin (PosOver).

④ ④ 참: 0 ~ TMax의 부호 반전은 모두 표현 가능.

⑤ ⑤ 거짓: x = TMin → −x = TMin < 0.

> ~TMin+1 = TMin

⑥ ⑥ 참: unsigned는 logical shift = floor = 정수 나눗셈.

⑦ ⑦ 거짓: x = −1 → −1 >> 3 = −1, −1/8 = 0 (음수에서 반올림 방향이 다름).

최종 답 **참: ②④⑥ · 거짓: ①(TMin) ③(TMax, 1) ⑤(TMin) ⑦(−1)**

---

2-7Endianness★★

little endian 기계에서 `int x = 0x1234ABCD`가 주소 0x200에 저장되어 있다. 0x200~0x203의 byte와, 같은 값이 big endian 기계에 저장된 경우를 쓰라. 또 `char s[] = "AB"` 는 어떻게 저장되는가?

힌트 보기

---

little = least significant byte가 lowest address. 문자열은 1바이트 단위라 endian과 무관.

정답 보기

---

① 바이트를 쪼갠다: 12 34 AB CD (MSB → LSB).

> LSB = CD

② little: LSB부터 낮은 주소에: 0x200 CD, 0x201 AB, 0x202 34, 0x203 12.

③ big: 사람이 읽는 순서대로: 0x200 12, 0x201 34, 0x202 AB, 0x203 CD.

④ "AB": 41 42 00. 문자 단위가 1바이트이므로 두 기계에서 동일.

최종 답 **little: CD AB 34 12 · big: 12 34 AB CD · "AB": 41 42 00 (동일)**